

Race Conditions Bug Bounty Guide

TOCTOU, HTTP/2 Single-Packet, Financial Exploitation & Tools

Double-Spend

Coupon Abuse

Limit Bypass

Inventory Abuse

May 2026

Table of Contents

- [1. What are Race Conditions](#)
- [2. How Race Conditions Work](#)
- [3. 50 Real-World Race Condition Issues](#)
- [4. TOCTOU Deep Dive](#)
- [5. HTTP/2 Single-Packet Attacks](#)
- [6. Tools & Automation](#)
- [7. Testing Methodology](#)
- [8. Report Templates](#)
- [9. Quick Reference](#)

1. What are Race Conditions

A race condition occurs when a system attempts to perform two or more operations simultaneously, and the outcome depends on the particular order in which the operations complete. In web applications, this means sending multiple requests at the exact same moment to exploit the tiny gap between when a server checks a condition and when it acts on that check.

The Check-Act Gap

Normal Flow (Safe):

1. Request 1: Check balance = \$100 → Balance ≥ \$100? YES
2. Request 1: Deduct \$100 → Balance = \$0
3. Request 2: Check balance = \$0 → Balance ≥ \$100? NO
4. Request 2: Reject transfer

Race Condition (Vulnerable):

1. Request 1: Check balance = \$100 → YES (at t=0ms)
2. Request 2: Check balance = \$100 → YES (at t=1ms, before update)
3. Request 1: Deduct \$100 → Balance = \$0
4. Request 2: Deduct \$100 → Balance = -\$100

Result: Two \$100 transfers from \$100 balance

Why Race Conditions are Dangerous

Aspect	Explanation
No scanner detection	Automated scanners cannot detect race conditions; requires manual testing
WAF bypass	Each request is legitimate; WAF sees normal traffic
High financial impact	Direct money/credit/asset theft
Hard to fix	Requires database-level locking, not simple patches
High bounty payouts	PI findings, \$2,000-\$20,000+

Severity by Impact

Race Type	Severity	Typical Bounty
Double-spend / Overdraft	P1	\$3,000 - \$20,000
Inventory oversell	P1	\$2,000 - \$10,000
Coupon/referral abuse	P1-P2	\$1,500 - \$10,000
Vote/like inflation	P2-P3	\$500 - \$3,000
Rate limit bypass	P2	\$500 - \$3,000
Registration abuse	P2-P3	\$300 - \$2,000

2. How Race Conditions Work

The TOCTOU Pattern

Time-of-Check to Time-of-Use is the fundamental pattern behind all race conditions:

Phase	Action	Vulnerability Window
Check	Server verifies: "Can this user do X?"	Condition is true
Window	Before server acts, another request arrives	Race occurs here
Act	Server performs action X	Both requests pass check

Types of Race Conditions

Type	Mechanism	Example
Single-object race	Multiple ops on same resource	Two withdrawals from same account
Multi-object race	Cross-resource inconsistency	Transfer from A while checking B balance
State transition race	State changes mid-operation	Cancel order after shipping started
Signal race	Notification vs action	Webhook fires before DB commits
Filesystem race	File operations	Temp file symlink attack

Concurrency Models Affected

Model	Vulnerable?	Why
Single-threaded	Yes (less)	Async I/O can still interleave
Multi-threaded	High	Threads interleave unpredictably
Multi-process	High	Shared state without locking
Microservices	Very High	Eventual consistency gaps
Serverless/Lambda	Very High	Cold starts, concurrency issues

Key Factors for Race Exploitation

Factor	Requirement	How to Achieve
Timing precision	Requests must arrive within milliseconds	HTTP/2 single-packet, Turbo Intruder
Request count	Multiple concurrent requests	20-100 parallel threads
Endpoint stability	Consistent processing time	Test multiple times, measure latency
No server-side locking	Database row not locked during check	Always assume no locking, test to confirm
Retry allowed	Failed requests can be retried	Most endpoints allow retries

Single-Packet Attack Explained

HTTP/2 allows multiplexing multiple requests over a single TCP connection. The **single-packet attack** sends multiple HTTP/2 requests in one TCP packet, eliminating network jitter:

```
HTTP/1.1 (Sequential, NOT vulnerable to race):
Request 1 ----> Server
          <---- Response 1
Request 2 ----> Server
          <---- Response 2

HTTP/2 Multiplexed (Race possible):
Request 1 + Request 2 + Request 3 ----> Server (single packet)
          <---- Response 1
          <---- Response 2
          <---- Response 3

All requests arrive at the EXACT same moment
```

3. 50 Real-World Race Condition Issues

Financial Race Conditions (#1-15)

P1 #1 Double Withdrawal from Wallet Balance

Target: Crypto/Wallet platforms – Binance, Coinbase style

Steps:

- Account balance: 1.0 BTC
- Prepare two withdrawal requests of 1.0 BTC each
- Send both simultaneously via Turbo Intruder (50 threads)
- Both withdrawals check balance (1.0 BTC >= 1.0) before either deducts
- Both withdrawals processed
- Balance: -1.0 BTC

Impact: Double-spending from single wallet balance

Ref: hackerone.com/reports/1186992, medium.com/@josipfranjic/race-double-withdrawal

P1 #2 Duplicate Transfer via Race Condition

Target: Banking/Transfer platforms – PayPal, Venmo style

Steps:

- Account balance: \$1000
- Send 20 transfer requests of \$1000 simultaneously
- All 20 pass balance check before any deduction
- 20 transfers of \$1000 each processed
- Account overdraft: -\$19,000

Impact: Transfer more than account balance

Ref: hackerone.com/reports/380102 (PayPal), medium.com/@josipfranjic/race-transfer

P1 #3 Credit Card Double Charge Refund Abuse

Target: Payment processing platforms

Steps:

- Purchase item for \$500 with credit card
- Send two refund requests simultaneously
- Both refunds processed before balance updates
- \$1000 refunded for \$500 purchase

Impact: Refund more than original payment

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-refund

P1 #4 Cryptocurrency Deposit Race (Same TXID)

Target: Crypto exchanges

Steps:

- Make deposit of 1 ETH with transaction hash 0xabc
- Submit deposit confirmation with same TXID 20 times
- All 20 processed before TXID marked as used
- 20 deposits of 1 ETH credited
- Balance inflated by 19 ETH

Impact: Same deposit credited multiple times

Ref: hackerone.com/reports/380102, [medium.com/@josipfranjic crypto-deposit-race](https://medium.com/@josipfranjic/crypto-deposit-race)

P1 #5 Staking Reward Double Claim

Target: DeFi/Crypto staking platforms

Steps:

- Have staking rewards of 100 tokens available
- Send 50 claim requests simultaneously
- All 50 pass available-rewards check
- $50 \times 100 = 5000$ tokens claimed
- Platform loses 4900 tokens

Impact: Staking rewards claimed multiple times

Ref: hackerone.com/reports/1186992, [medium.com/@josipfranjic staking-race](https://medium.com/@josipfranjic/staking-race)

P1 #6 Gift Card Balance Transfer Race

Target: Gift card platforms - Amazon, Apple style

Steps:

- Gift card A balance: \$100
- Send two transfer requests: A→B (\$100) simultaneously
- Both check balance ($\$100 \geq \100) before deduction
- Both transfers succeed, B gets \$200, A goes negative

Impact: Gift card balance duplicated via transfer race

Ref: hackerone.com/reports/403050, [medium.com/@bugbounty race-gift-card](https://medium.com/@bugbounty/race-gift-card)

P1 #7 Coupon Stacking via Race Condition

Target: E-commerce - Generic

Steps:

- Add item (\$100) to cart
- Apply "50OFF" coupon
- Send 10 checkout requests simultaneously with same coupon
- All 10 checkouts process before coupon marked used
- 10 orders at \$50 each, coupon used 10 times

Impact: Single-use coupon applied multiple times

Ref: hackerone.com/reports/351344, [medium.com/@seanmelia race-coupon](https://medium.com/@seanmelia/race-coupon)

P1 #8 Referral Bonus Multiplication

Target: Referral systems - Uber, Dropbox style

Steps:

- Send referral link to new user
- New user signs up
- Send 20 bonus-credit requests simultaneously
- All 20 check if bonus credited (no) before marking
- $20 \times \$10 = \200 bonus for one referral

Impact: Referral bonus credited multiple times

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-referral

P1 #9 Welcome Bonus Multiple Credit

Target: Sign-up bonus platforms

Steps:

- Create account
- Send 20 welcome-bonus requests simultaneously
- All 20 processed before account marked as bonus-received
- $20 \times \$5 = \100 welcome bonus

Impact: Welcome bonus multiplied via race

Ref: hackerone.com/reports/237166, medium.com/@anshuman_bh/race-welcome-bonus

P2 #10 Subscription Cancellation Timing for Full Refund

Target: SaaS subscription platforms

Steps:

- Subscribe to annual plan (\$1000)
- Use service for 11 months
- Cancel subscription at exact billing moment
- Race between usage and refund calculation
- Full \$1000 refund despite 11 months usage

Impact: Full refund after extended usage

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-subscription-refund

P1 #11 Negative Balance via Race Condition

Target: Wallet/Banking platforms

Steps:

- Account balance: \$10
- Send two purchase requests of \$10 simultaneously
- Both pass balance check ($\$10 \geq \10)
- Both processed, balance: $-\$10$
- Negative balance without overdraft protection

Impact: Create negative balance via race

Ref: hackerone.com/reports/380102, medium.com/@josipfranjic/race-negative-balance

P1 #12 Interest Payment Race on Savings

Target: Banking/Interest platforms

Steps:

- Have savings account eligible for interest
- Interest calculated daily at midnight
- Send 50 withdrawal+deposit cycles at midnight
- Race between interest calc and balance updates
- Interest calculated on wrong (inflated) balance

Impact: Incorrect interest calculation via timing

Ref: hackerone.com/reports/286048, [medium.com/@detectify race-interest](https://medium.com/@detectify/race-interest)

P2 #13 Fee Waiver via Race (Multiple Waivers)

Target: Banking/Transaction fee platforms

Steps:

- One fee waiver per month
- Send 10 fee waiver requests simultaneously
- All 10 processed before waiver counter updates
- 10 fee waivers applied in one month

Impact: Multiple fee waivers via race

Ref: hackerone.com/reports/189234, [medium.com/@busesz race-fee-waiver](https://medium.com/@busesz/race-fee-waiver)

P1 #14 Insurance Claim Double Processing

Target: Insurance platforms

Steps:

- Submit insurance claim for \$5000
- Send two claim processing requests
- Both processed before claim marked as paid
- Two \$5000 payouts for one claim

Impact: Duplicate insurance payouts

Ref: hackerone.com/reports/286091, [medium.com/@seanmelia race-insurance](https://medium.com/@seanmelia/race-insurance)

P2 #15 Currency Conversion Rate Lock Race

Target: Forex/Crypto conversion

Steps:

- Lock favorable conversion rate
- Rate changes unfavorably
- Race: send conversion request at old rate while rate updates
- Conversion processed at old favorable rate
- Profit from rate discrepancy

Impact: Exploit rate changes via timing race

Ref: hackerone.com/reports/380102, [medium.com/@josipfranjic race-forex](https://medium.com/@josipfranjic/race-forex)

Inventory & Resource Abuse (#16-30)

P1 #16 Limited Item Oversell (1 Item -> 50 Orders)

Target: E-commerce limited drops - Supreme, NFT drops

Steps:

- Limited item: 1 in stock
- 50 users (or 50 requests) add to cart simultaneously
- All 50 checkouts process before inventory decrements
- 50 orders confirmed for 1 item
- Platform must cancel 49 orders or lose money

Impact: Sell more inventory than available

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-inventory-oversell

P1 #17 Ticket Booking Race (Same Seat Multiple Times)

Target: Ticketing - Eventbrite, Ticketmaster style

Steps:

- Concert has 100 seats
- 200 users select Seat A1 simultaneously
- All 200 pass availability check before reservation
- 200 bookings for 1 seat
- Platform overbooks event

Impact: Overbook events via seat reservation race

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-ticket-booking

P1 #18 Hotel Room Overbooking Race

Target: Travel - Booking.com, Airbnb style

Steps:

- Hotel has 1 room available
- 5 users book the room simultaneously
- All 5 pass availability check
- 5 confirmed bookings for 1 room
- Platform overbooks

Impact: Hotel overbooking via race condition

Ref: hackerone.com/reports/189234, medium.com/@busesz/race-hotel

P1 #19 Discount Code Single-Use Bypass

Target: E-commerce discount systems

Steps:

- Single-use discount: SAVE50
- Add item to cart, apply SAVE50
- Send 20 checkout requests simultaneously

- All 20 checkouts process before code marked used
- 20 orders with SAVE50 applied

Impact: Single-use code used 20 times

Ref: hackerone.com/reports/417714, medium.com/@anshuman_bh/race-discount-code

P2 #20 API Rate Limit Bypass via Burst

Target: API platforms – Twitter API, Google API style

Steps:

- Rate limit: 100 requests/minute
- Send 500 requests in first second using parallel connections
- All 500 processed before rate counter updates
- API abused 5x over limit

Impact: Bypass API rate limiting

Ref: hackerone.com/reports/1176994, medium.com/@nahamsec/race-ratelimit

P2 #21 Storage Quota Bypass via Parallel Uploads

Target: Cloud storage – Google Drive, Dropbox style

Steps:

- Storage limit: 10GB
- Upload 20 files of 1GB simultaneously
- All 20 processed before quota check
- 20GB uploaded to 10GB account
- Quota exceeded without enforcement

Impact: Upload beyond storage limits

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-storage-quota

P1 #22 Voucher/Redemption Code Multiple Use

Target: Voucher systems

Steps:

- Redeem voucher code: GIFT-100 for \$100 credit
- Send 10 redemption requests simultaneously
- All 10 processed before code marked redeemed
- \$1000 credited from single voucher

Impact: Voucher redeemed multiple times

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-voucher

P1 #23 Credit Limit Bypass (Multiple Simultaneous Purchases)

Target: Credit/BNPL platforms – Klarna, Afterpay style

Steps:

- Credit limit: \$500
- Send 10 purchase requests of \$500 simultaneously

- All 10 pass credit check before limit updates
- \$5000 purchased on \$500 limit

Impact: Exceed credit limit via race

Ref: hackerone.com/reports/380102, medium.com/@josipfranjic/race-credit-limit

P2 #24 Free Trial Abuse via Parallel Account Creation

Target: SaaS trial platforms

Steps:

- Free trial: 1 per email
- Send 10 registration requests with same email simultaneously
- All 10 processed before uniqueness check
- 10 accounts created with same email, 10 free trials

Impact: Multiple free trials from one email

Ref: hackerone.com/reports/237166, medium.com/@anshuman_bh/race-trial

P2 #25 Reservation Hold Abuse (Hold Multiple Slots)

Target: Restaurant/appointment booking

Steps:

- Max 1 reservation per user
- Send 5 reservation requests for different time slots simultaneously
- All 5 processed before reservation counter updates
- 5 reservations when only 1 allowed

Impact: Multiple reservations via race

Ref: hackerone.com/reports/286091, medium.com/@seanmelia/race-reservation

P1 #26 Inventory Transfer Race (Duplicate Items)

Target: Warehouse/Inventory management

Steps:

- Warehouse A has 1 item
- Send two transfer requests: A->B simultaneously
- Both pass inventory check ($1 \geq 1$)
- Two transfers processed, B receives 2, A goes negative

Impact: Duplicate inventory transfers

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-inventory-transfer

P2 #27 Seat Reservation Hold Without Release

Target: Airline/Train booking

Steps:

- Reserve seat (hold for 10 minutes)
- Send 100 reservation requests for different seats
- All held without release mechanism

- All seats blocked, no actual booking
- Denial of inventory

Impact: Block all inventory without purchase

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-seat-hold

P2 #28 Promo Code First-Come Abuse

Target: Limited promo campaigns

Steps:

- Promo: First 100 users get \$50 off
- Send 500 requests simultaneously
- 500 users get \$50 off despite 100 limit
- Promo budget exceeded 5x

Impact: Promo limit bypass via race

Ref: hackerone.com/reports/189234, medium.com/@busesz/race-promo

P2 #29 Download Token Reuse via Race

Target: Digital content platforms

Steps:

- Single-use download token for premium content
- Send 10 download requests with same token
- All 10 processed before token invalidated
- 10 downloads from single token

Impact: Single-use token used multiple times

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-download-token

P2 #30 Resource Pool Exhaustion via Parallel Allocation

Target: Cloud resource platforms – AWS, GCP style

Steps:

- Resource limit: 10 VMs per account
- Send 50 VM creation requests simultaneously
- All 50 processed before quota check
- 50 VMs created on 10-VM limit

Impact: Exceed resource allocation limits

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-resource-pool

Social & Engagement Abuse (#31-42)

P2 #31 Vote Manipulation (1 Vote -> 100 Votes)

Target: Voting platforms – Twitter polls, Reddit style

Steps:

- One vote per user limit

- Send 100 vote requests simultaneously
- All 100 processed before vote counter updates
- 100 votes counted from 1 user

Impact: Inflated vote counts in polls

Ref: hackerone.com/reports/189234, medium.com/@busesz/race-vote-100x

P2 #32 Like/Heart Inflation via Race

Target: Social platforms - Instagram, Twitter style

Steps:

- Like a post
- Send 50 like requests simultaneously
- All 50 processed before like counter updates
- 50 likes from single account
- Engagement metrics inflated

Impact: Inflated engagement metrics

Ref: hackerone.com/reports/237166, medium.com/@anshuman_bh/race-like-inflation

P2 #33 Follower/Subscriber Count Inflation

Target: Social platforms - YouTube, Twitch style

Steps:

- Follow a user
- Send 50 follow requests simultaneously
- All 50 processed before follow relationship check
- 50 follows registered from 1 account
- Follower count artificially inflated

Impact: Fake follower count via race

Ref: hackerone.com/reports/237166, medium.com/@anshuman_bh/race-follower

P2 #34 Friend Request Spam via Race

Target: Social platforms - Facebook, LinkedIn style

Steps:

- Send 1000 friend requests simultaneously
- All processed before rate limiting kicks in
- 1000 friend requests sent instantly
- Spam/bot behavior

Impact: Friend request spam bypassing rate limits

Ref: hackerone.com/reports/195286, medium.com/@knownsec/race-friend-spam

P2 #35 Comment/Review Multiple Posting

Target: Review platforms - Yelp, Amazon style

Steps:

- One review per product limit
- Send 10 review submissions simultaneously
- All 10 processed before duplicate check
- 10 reviews posted from 1 account

Impact: Multiple reviews from single account

Ref: hackerone.com/reports/189234, medium.com/@busesz/race-review-spam

P2 #36 Referral Code Assignment Race (Steal Referrals)

Target: Referral platforms

Steps:

- User clicks referral link
- Send 10 referral-assignment requests with different codes
- Race between referral codes
- Wrong referrer credited

Impact: Referral attribution stolen

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-referral-steal

P2 #37 Badge/Achievement Multiple Award

Target: Gaming/Community platforms

Steps:

- Achievement: Complete 1 task
- Send 10 achievement-claim requests simultaneously
- All 10 processed before achievement check
- Achievement points credited 10 times

Impact: Achievement points multiplied

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-achievement

P1 #38 Points/Credit Multiplication via Race

Target: Loyalty/Rewards platforms

Steps:

- Earn 100 points for action X
- Perform action X, send 20 point-claim requests
- All 20 processed before point-award check
- 2000 points from single action

Impact: Loyalty points multiplied

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-points

P2 #39 Reputation/Karma Inflation

Target: Community platforms – Reddit, StackOverflow style

Steps:

- Upvote a post (+1 karma)

- Send 50 upvote requests simultaneously
- All 50 processed before karma counter updates
- +50 karma from single upvote

Impact: Inflated reputation scores

Ref: hackerone.com/reports/237166, medium.com/@anshuman_bh race-karma

P2 #40 Share/Retweet Count Inflation

Target: Social platforms - Twitter style

Steps:

- Retweet a post
- Send 30 retweet requests simultaneously
- All 30 processed before retweet counter updates
- 30 retweets counted from 1 account

Impact: Inflated share counts

Ref: hackerone.com/reports/487954 (Twitter), medium.com/@nahamsec race-retweet

Account & Authentication Abuse (#41-45)

P2 #41 Account Creation Limit Bypass (Same Email Multiple Accounts)

Target: Registration systems

Steps:

- Limit: 1 account per email
- Send 5 registration requests with same email simultaneously
- All 5 processed before uniqueness check
- 5 accounts created with same email

Impact: Multiple accounts with same email

Ref: hackerone.com/reports/237166, medium.com/@anshuman_bh race-reg-limit

P2 #42 2FA Code Validation Race (Guess OTP in Bulk)

Target: 2FA enabled platforms

Steps:

- 4-digit OTP required
- Send all 10000 combinations in parallel
- Processing race between guesses
- Correct OTP validated before rate limit
- 2FA bypassed via brute force race

Impact: 2FA bypass via parallel brute force

Ref: hackerone.com/reports/1166056, medium.com/@nahamsec race-2fa-brute

P2 #43 Password Reset Token Brute Force via Race

Target: Password reset flows

Steps:

- 5-digit reset token
- Send 100000 token guesses simultaneously
- Race between guesses and rate limiting
- Correct token found before lockout

Impact: Reset token brute forced via race

Ref: hackerone.com/reports/170831, medium.com/@nahamsec/race-reset-brute

P3

#44 Session Creation Race (Same Credentials Multiple Sessions)

Target: Session management

Steps:

- Login with credentials
- Send 50 login requests simultaneously
- All 50 processed before session deduplication
- 50 active sessions for 1 account

Impact: Multiple sessions from single login

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-session

P2

#45 Account Merge Race (Merge Multiple Accounts)

Target: Account management

Steps:

- Account A and B eligible for merge
- Send 3 merge requests: A→B, A→C, B→C simultaneously
- Race between merge operations
- Accounts merged inconsistently
- Data corruption or unauthorized merge

Impact: Account merge inconsistency

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-account-merge

API & System Abuse (#46-50)

P2

#46 Webhook Delivery Race (Duplicate Webhook Events)

Target: Webhook-based platforms – Stripe, GitHub style

Steps:

- Event triggers webhook
- Race between webhook sender and event status update
- Same webhook delivered 3 times
- Downstream system processes 3x

Impact: Duplicate webhook events processed

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-webhook

P2 #47 Email Bombing via Race (Bypass Rate Limit)

Target: Email notification systems

Steps:

- Rate limit: 1 email per minute
- Send 1000 email trigger requests simultaneously
- All 1000 processed before rate counter updates
- 1000 emails sent instantly

Impact: Email spam via race condition

Ref: hackerone.com/reports/195286, medium.com/@knownsec/race-email-bomb

P2 #48 SMS Bombing via Race (Bypass SMS Rate Limit)

Target: SMS verification systems

Steps:

- Rate limit: 1 SMS per 5 minutes
- Send 500 SMS trigger requests simultaneously
- All 500 processed before rate counter updates
- 500 SMS sent to victim phone

Impact: SMS bombing via race

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-sms-bomb

P1 #49 Order Status Confusion Race (Cancel After Ship)

Target: E-commerce order management

Steps:

- Place order, status: PAID
- Order ships, status changes to SHIPPED
- Race: send cancel request at exact ship moment
- Cancel processed after ship but before status update
- Order cancelled but already shipped
- Customer gets free item

Impact: Free items via order cancellation race

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-order-cancel

P1 #50 Database State Corruption via Concurrent Updates

Target: Generic multi-user platforms

Steps:

- User A edits field X to value A
- User B edits field X to value B
- Both edits processed simultaneously
- Database ends with inconsistent state: field X = mix of A and B
- Or: update lost (last write wins incorrectly)

Impact: Data corruption via concurrent writes

Ref: hackerone.com/reports/286048, medium.com/@detectify/race-db-corruption

4. TOCTOU Deep Dive

TOCTOU in Financial Operations

Operation	Check	Use	Exploit
Withdrawal	balance >= amount	Deduct balance	Two withdrawals between check and deduct
Transfer	sender.balance >= amount	Debit sender, credit receiver	Transfer to multiple receivers from same balance
Coupon redeem	coupon.used == false	Mark coupon used	Multiple redemptions before marking
Inventory check	stock > 0	Decrement stock	Buy more than stock
Credit check	credit_used < credit_limit	Increment credit_used	Exceed credit limit
Rate limit	requests < limit	Increment counter	Send burst exceeding limit
One-time action	action_done == false	Mark action done	Repeat action before marking
Unique constraint	email not exists	Insert user	Create duplicate accounts
Reservation	seat available	Mark seat reserved	Book same seat multiple times
Token validation	token valid and unused	Mark token used	Use token multiple times

TOCTOU Code Examples

```
// VULNERABLE Python (Django):
# views.py
def withdraw(request):
    amount = request.POST['amount']
    user = request.user
    # CHECK
    if user.balance >= amount: # ← Check here
        # GAP: Another request can pass check here!
        # USE
        user.balance -= amount    # ← Use here
        user.save()
        return JsonResponse({"status": "success"})

// SECURE (using F() expression for atomic update):
from django.db.models import F
def withdraw(request):
    amount = request.POST['amount']
    # Atomic: check and update in single DB operation
    updated = User.objects.filter(
        id=request.user.id,
        balance__gte=amount
    ).update(balance=F('balance') - amount)
```

```
return JsonResponse({"status": "insufficient_funds"})
return JsonResponse({"status": "success"})
```

TOCTOU in Different Database Systems

Database	Solution	Code Pattern
PostgreSQL	SELECT ... FOR UPDATE	BEGIN; SELECT balance FROM accounts WHERE id=1 FOR UPDATE; UPDATE ...; COMMIT;
MySQL (InnoDB)	SELECT ... FOR UPDATE	Same as PostgreSQL with row-level locking
MongoDB	findAndModify()	db.accounts.findAndModify({query: {balance: {\$gte: amount}}, update: {\$inc: {balance: -amount}}})
Redis	Atomic operations	WATCH balance; GET balance; MULTI; SET balance new_val; EXEC;
Django ORM	F() expressions	User.objects.filter(id=1, balance__gte=amount).update(balance=F('balance')-amount)

Key Insight: The gap between check and use is typically 1-50 milliseconds. In that window, a second request can pass the same check. The fix is to make check and use atomic using database locks or compare-and-swap operations.

5. HTTP/2 Single-Packet Attacks

Why HTTP/2 Changes the Game

HTTP Version	Request Model	Race Reliability
HTTP/1.1	Sequential per connection	Low - requests arrive milliseconds apart
HTTP/1.1 Pipelining	Multiple requests in flight	Medium - still separate frames
HTTP/2 Multiplexed	Multiple streams, single connection	High - requests in same packet
HTTP/2 Single-Packet	Multiple streams in one TCP packet	Very High

HTTP/2 Frame Structure for Race Attacks

HTTP/2 frames sent in ONE TCP packet:

```
+-----+-----+-----+-----+-----+-----+
| HEADERS (stream 1) - Request 1          |
+-----+-----+-----+-----+-----+-----+
| HEADERS (stream 3) - Request 2          |
+-----+-----+-----+-----+-----+-----+
| HEADERS (stream 5) - Request 3          |
+-----+-----+-----+-----+-----+-----+
| HEADERS (stream 7) - Request 4          |
+-----+-----+-----+-----+-----+-----+
| DATA (stream 1) - Body 1                |
+-----+-----+-----+-----+-----+-----+
| DATA (stream 3) - Body 2                |
+-----+-----+-----+-----+-----+-----+
```

All streams processed by server simultaneously

Creating Single-Packet Attacks

Method	Tool	Command
Turbo Intruder	Burp Extension	Engine.THREADED with gate synchronization
Last-byte sync	Custom Python	Pause before sending last byte of each request
Single-packet race	race.py	python race.py -u URL -r req.txt -n 20
HTTP/2 tool	h2race	h2race -u URL -c 20 --synchronize
Custom socket	Python raw	Build HTTP/2 frames manually, send in one write()

Last-Byte Synchronization Technique

```
import socket, time, threading

HOST = 'target.com'
PORT = 443
```

```

def send_partial(sock, request):
    # Send all but last byte
    sock.sendall(request[:-1])

def sync_attack():
    sock = socket.create_connection((HOST, PORT))
    # TLS handshake here...

    requests = [build_http2_request(f"/transfer?amount=100") for _ in range(NUM_REQUESTS)]

    threads = []
    for req in requests:
        t = threading.Thread(target=send_partial, args=(sock, req))
        threads.append(t)
        t.start()

    # All threads sent n-1 bytes
    time.sleep(0.1) # Ensure all partial sends complete

    # NOW send all last bytes simultaneously
    for req in requests:
        sock.sendall(req[-1:])

# Result: All 20 requests arrive at server in same microsecond

```

Server-Side Indicators of Single-Packet Success

Indicator	Meaning
All responses have same timestamp	Requests processed simultaneously
Response times are nearly identical	< 1ms variance between responses
Some responses succeed, some fail	Race occurred (inconsistent results)
Database constraint error on some	Some won the race, others failed late
All responses succeed (unexpected)	Race condition confirmed (no locking)

6. Tools & Automation

Turbo Intruder (Burp Suite)

Turbo Intruder is the most effective tool for race condition testing in web applications.

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
        concurrentConnections=30,
        requestsPerConnection=100,
        pipeline=False,
        engine=Engine.THREADED)
    for i in range(20):
        engine.queue(target.req, target.baseInput, gate="race1")
    engine.openGate("race1")

def handleResponse(req, interesting):
    table.add(req)
```

Race Condition Python Script

```
import asyncio, aiohttp, time

async def race_request(session, url, payload, results):
    async with session.post(url, data=payload) as resp:
        text = await resp.text()
        results.append((resp.status, text[:100]))

async def race_attack(url, payload, count=20):
    results = []
    async with aiohttp.ClientSession() as session:
        tasks = [race_request(session, url, payload, results)
            for _ in range(count)]
        start = time.time()
        await asyncio.gather(*tasks)
        elapsed = time.time() - start
        success_count = sum(1 for s, _ in results if s == 200)
        print(f"Success: {success_count}/{count} in {elapsed:.3f}s")
        if success_count > 1:
            print("[+] RACE CONDITION CONFIRMED!")
        return results

# Usage: asyncio.run(race_attack("https://t/api/withdraw",
#                                {"amount": 100}, count=30))
```

Bash Parallel Race

```
# GNU parallel:
parallel -j20 curl -s -X POST -H "Auth: Bearer $TOKEN"
-d '{"amount":100}' https://t/api/withdraw ::: $(seq 1 20)

# xargs:
seq 1 20 | xargs -P20 -I{} curl -s -X POST
```



```
https://t/api/withdraw
```

```
# Background jobs:
for i in {1..20}; do
  curl -s -X POST -H "Auth: Bearer $TOKEN"
    -d '{"amount":100}' https://t/api/withdraw &
done; wait
```

HTTP/2 Race with httpx

```
import httpx, asyncio

async def h2_race(url, payload, count=20):
    limits = httpx.Limits(max_connections=count)
    async with httpx.AsyncClient(http2=True,
                                limits=limits) as client:
        tasks = [client.post(url, data=payload)
                  for _ in range(count)]
        responses = await asyncio.gather(*tasks)
        return [r.status_code for r in responses]

# Check HTTP/2: curl -I --http2 https://target.com
```

Tool Comparison

Tool	Best For	Speed	HTTP/2	Setup
Turbo Intruder	Advanced races, gates	Very Fast	Yes	Burp Plugin
asyncio + aiohttp	Custom scripts	Fast	No	Easy
httpx HTTP/2	HTTP/2 races	Fast	Yes	Easy
Burp Intruder	Simple races	Medium	No	Built-in
GNU parallel	Quick bash	Medium	No	Built-in
Custom sockets	Single-packet	Fastest	Yes	Complex

7. Testing Methodology

Phase 1: Identify Race-Prone Endpoints

Indicator	Why It is Race-Prone	Example Endpoint
Check-then-act pattern	Balance check before deduction	/api/withdraw, /api/transfer
One-time operation	Single use enforced by flag	/api/coupon/redeem, /api/referral/claim
Limited resource	Inventory or quota check	/api/purchase, /api/book
Unique constraint	Duplicate prevention	/api/register, /api/create
Counter increment	Usage counting	/api/vote, /api/like
State transition	Multi-step state machine	/api/order/confirm, /api/cancel
Credit/debit flow	Financial calculation	/api/refund, /api/deposit
Rate limiting	Request counting	/api/send, /api/notify

Phase 2: Baseline Testing

Step	Action	Expected Result
1	Send single request, note response time	Baseline: ~100-500ms
2	Send 2 parallel requests	Both process, check for inconsistencies
3	Send 5 parallel requests	Look for partial failures
4	Send 20 parallel requests	Race condition likely if no locking
5	Verify state change	Check if more actions succeeded than should

Phase 3: Confirmation Testing

Test	How	Confirmation
Balance verification	Check balance before and after	Balance should be X but is Y
Count verification	Count affected records	More records than allowed
Repeatability	Run test 5 times	Race should trigger at least once
Sequential vs parallel	Send 10 sequential then 10 parallel	Sequential all pass or fail correctly; parallel has inconsistencies
Consistency check	Verify database state	Database in inconsistent state

Phase 4: Impact Assessment

Impact Level	Criteria	Example
--------------	----------	---------

Critical	Financial loss, direct money theft	Double withdrawal exceeding balance
High	Business logic bypass, resource abuse	Inventory oversell, coupon abuse
Medium	Rate limit bypass, spam	Vote inflation, email bombing
Low	Minor inconsistency	Session count off by 1

Testing Order: 1) Find endpoints with check-then-act 2) Test with 2 parallel requests 3) Scale to 20-50 4) Verify state 5) Document inconsistency. Start with financial endpoints – they pay the most.

8. Report Templates

Template: Financial Race Condition (Double-Spend)

Section	Content
Title	Race Condition: Double Withdrawal Exceeding Account Balance
Severity	Critical (CVSS 3.1: 8.9)
Description	The /api/withdraw endpoint checks account balance before processing withdrawal but does not lock the balance during the operation. Sending two withdrawal requests simultaneously allows both to pass the balance check before either deduction is applied.
Steps	1. Account balance: \$100 2. Send two POST /api/withdraw requests of \$100 simultaneously (20 threads) 3. Both requests pass balance check ($\\$100 \geq \\100) 4. Both withdrawals processed 5. Final balance: $-\\$100$
Impact	Users can withdraw more than their balance. Direct financial impact - \$200 withdrawn from \$100 balance.
Proof	[Screenshot: Balance before \$100] [Screenshot: Turbo Intruder 20 requests] [Screenshot: Two 200 OK responses] [Screenshot: Balance after $-\$100$]
Fix	Use SELECT ... FOR UPDATE (row-level locking) or atomic F() expressions (Django) to ensure balance check and deduction occur atomically.

Template: Inventory Oversell Race

Section	Content
Title	Race Condition: Limited Item Oversell - 50 Orders from 1 Unit Stock
Severity	Critical (CVSS 3.1: 8.2)
Description	The /api/purchase endpoint checks inventory before confirming order but does not lock the inventory row during purchase processing.
Steps	1. Limited item: 1 in stock 2. 50 users add to cart 3. All 50 send checkout simultaneously 4. All 50 pass inventory check ($\text{stock}=1 > 0$) 5. All 50 orders confirmed 6. 50 orders for 1 item
Impact	50 orders confirmed for 1 item. Platform must cancel 49 orders or fulfill at loss.
Fix	Use database row locking during inventory check and decrement as atomic operation.

Template: Rate Limit Bypass

Section	Content
Title	API Rate Limit Bypass via Request Race Condition
Severity	High (CVSS 3.1: 7.5)
Description	API rate limiting is enforced via in-memory counter that increments after processing. Sending a burst of requests bypasses the counter.
Steps	1. Rate limit: 100/minute 2. Send 500 requests in 1 second 3. All 500 processed before counter updates 4. All 500 responses returned
Impact	5x rate limit bypassed. Enables DoS, brute force, API abuse.
Fix	Use distributed rate limiting with Redis (INCR + EXPIRE) or check rate before processing.

Bounty Reference

Finding	Platform	Bounty Range
Double-spend / Financial race	HackerOne	\$3,000 - \$20,000
Double-spend	Bugcrowd	\$2,000 - \$15,000
Inventory oversell	HackerOne	\$2,000 - \$10,000
Coupon race abuse	HackerOne	\$1,500 - \$8,000
Rate limit bypass	HackerOne	\$500 - \$3,000
Vote/like inflation	HackerOne	\$300 - \$2,000

9. Quick Reference

Common Race-Prone Endpoints

Category	Endpoints	Check For
Financial	/withdraw, /transfer, /refund, /deposit, /pay	Balance check before deduction
E-commerce	/checkout, /purchase, /order, /apply-coupon	Inventory/coupon check before use
Booking	/book, /reserve, /schedule, /appointment	Availability check before hold
Rewards	/claim, /redeem, /refer, /reward	One-time flag check before credit
Social	/vote, /like, /follow, /share	Count check before increment
Auth	/register, /login, /verify-otp	Unique/one-time constraint check
API	/api/* (rate limited)	Counter check before increment
Communication	/send, /notify, /email, /sms	Rate limit check before send

Quick Start Commands

Purpose	Command
Quick race test (bash)	for i in {1..20}; do curl -s -X POST -H "Auth: \$T" -d '{"a":100}' URL & done; wait
Quick race test (GNU parallel)	parallel -j20 curl -s -X POST -H "Auth: \$T" -d '{"a":100}' URL ::: \$(seq 1 20)
Turbo Intruder race	Engine.THREADED, concurrentConnections=30, gate="race1"
Python asyncio race	asyncio.gather(*[client.post(URL, data=payload) for _ in range(20)])
HTTP/2 race	httpx.AsyncClient(http2=True) + asyncio.gather 20 requests
Balance check	GET /balance before and after race, compare

Indicators of Vulnerability

Signal	Meaning
Multiple 200 OK responses	All requests succeeded (potential race)
Mixed 200/400 responses	Some won the race, others lost
Same timestamp on responses	Processed simultaneously
Database constraint errors	Race detected late by DB
Negative balance/quantity	Confirmed financial race
Inconsistent state	Data corruption from race

Testing Checklist

- ☐ Identify endpoints with check-then-act pattern

- ☐ Test financial endpoints (withdraw, transfer, refund)
- ☐ Test inventory endpoints (purchase, book, reserve)
- ☐ Test one-time operations (coupon, referral, reward)
- ☐ Test rate-limited endpoints
- ☐ Test unique constraint endpoints (register)
- ☐ Send 2 parallel requests first
- ☐ Scale to 20 parallel requests
- ☐ Try HTTP/2 multiplexing
- ☐ Verify state after race (balance, count, status)
- ☐ Test 5 times for repeatability
- ☐ Document before/after state
- ☐ Capture all response codes and bodies
- ☐ Calculate financial impact if applicable
- ☐ Report with reproduction steps and proof

Prevention for Developers

Pattern	Vulnerable	Secure
Check then update	if balance >= amount: balance - = amount	UPDATE accounts SET balance = balance - amount WHERE balance >= amount
Read then write	count = get_count(); set_count(count + 1)	UPDATE counters SET count = count + 1 WHERE id = 1
Check flag then set	if not used: mark_used()	UPDATE coupons SET used = true WHERE id = X AND used = false
Check unique then insert	if not exists: insert()	INSERT IGNORE or UNIQUE constraint
Read inventory then sell	if stock > 0: stock -= 1	UPDATE inventory SET stock = stock - 1 WHERE stock > 0

Remember: Race conditions are about timing. The more parallel requests you send, the higher the chance of hitting the vulnerability window. Always test with at least 20 parallel requests. Financial endpoints are the highest-paying targets – prioritize them.